

Caderno de Desafios Rápidos

Aula 01 — Introdução à Orientação a Objetos

Este caderno utiliza duas técnicas científicas de aprendizagem: Prática de Recuperação (recuperar ativamente é mais eficaz do que reler) e Repetição Espaçada (revisitar conceitos anteriores fortalece a memória de longo prazo). Tente responder cada desafio sem consultar o material — depois confira o gabarito no final.

■ **Tempo sugerido: 25-30 minutos** | ■ **Formato: individual ou em duplas** | ■ **Gabarito: páginas finais**

SEÇÃO 1 — REVISÃO ESPAÇADA | Pré-requisitos (Módulos anteriores) | 5 mi

Recupere o que você já aprendeu nos módulos de Fundamentos!

Desafio 1 [Revisão]

Qual a diferença entre os tipos primitivos `int` e `double` em Java? Dê um exemplo de quando cada um seria a melhor escolha.

Dica: Módulo 01 — tipos numéricos e precisão decimal.

Desafio 2 [Revisão]

Escreva um trecho de código Java que leia o nome do usuário pelo teclado e imprima 'Olá, NOME!' na tela. Pode usar `Scanner`.

Dica: Lembre-se de importar `java.util.Scanner` e fechar o `Scanner`.

Desafio 3 [Revisão]

Considere o trecho abaixo. O que ele imprime e por quê?

```
int x = 10;
if (x > 5 && x < 20) System.out.println("OK");
else System.out.println("FORA");
```

Dica: Operadores relacionais e lógicos retornam `true` ou `false`.

Desafio 4 [Revisão]

Escreva um laço `for` que imprima os números pares de 0 a 20 (inclusive).

Dica: Pode incrementar de 2 em 2 ou usar o operador `%` para testar paridade.

SEÇÃO 2 — CONTEÚDO DA AULA: BÁSICO | Classes, objetos, métodos | 10 m

Teste o que você acabou de aprender!

Desafio 5 [Básico]

Verdadeiro ou Falso? Justifique cada resposta:

- a) Uma classe ocupa memória mesmo sem ser instanciada.
- b) Dois objetos da mesma classe podem ter valores diferentes nos atributos.
- c) O operador new é obrigatório para criar um objeto.
- d) Sem construtor explícito, é impossível instanciar uma classe.

Dica: Pense em planta x casa, e em construtor padrão.

Desafio 6 [Básico]

Declare uma classe Livro com dois atributos (titulo e paginas) e dois métodos (getTitulo e getPaginas). Não precisa de construtor por enquanto — só a estrutura básica.

Dica: Lembre-se do padrão: PascalCase no nome da classe, camelCase nos métodos.

Desafio 7 [Básico]

Considere a classe abaixo. Escreva o código no main que cria DOIS objetos do tipo Cachorro com nomes diferentes e chama o método latir() em ambos.

```
public class Cachorro {  
    String nome;  
    public void latir() {  
        System.out.println(nome + " disse Au!");  
    }  
}
```

Dica: Cada objeto guarda seu próprio nome — atribua via dot notation.

Desafio 8 [Básico]

O código abaixo não compila. Identifique o erro e explique a correção:

```
public class Aluno {  
    String nome;  
}  
// no main:  
Aluno a;  
a.nome = "Maria";  
System.out.println(a.nome);
```

Dica: O que falta entre a declaração e o uso?

SEÇÃO 3 — CONTEÚDO DA AULA: INTERMEDIÁRIO | Construtores, encapsulamento

Desafie seu raciocínio!

Desafio 9 [Intermediário]

Crie a classe `ContaBancaria` com os atributos `numero` (`String`) e `saldo` (`double`), ambos privados. Adicione um construtor que recebe o número da conta e inicializa o saldo com 0.0. Adicione também os métodos `depositar(double valor)` e `getSaldo()`.

Dica: private nos atributos, public no construtor e nos métodos.

Desafio 10 [Intermediário]

O que o código abaixo imprime? Justifique passo a passo:

```
public class Lampada {
    private boolean ligada = false;
    public void ligar() { ligada = true; }
    public void apagar() { ligada = false; }
    public boolean isLigada() { return ligada; }
}
// main:
Lampada l = new Lampada();
l.ligar(); l.ligar(); l.apagar();
System.out.println(l.isLigada());
```

Dica: O estado é apenas o resultado da última alteração.

Desafio 11 [Intermediário]

Implemente o setter `setIdade(int idade)` da classe `Pessoa`, mas com validação: se a idade for menor que 0 ou maior que 130, deve recusar o valor (lançar `IllegalArgumentException` ou simplesmente ignorar — escolha e justifique).

Dica: Esse é o motivo de usarmos setters em vez de acesso direto ao atributo.

Desafio 12 [Intermediário]

O código abaixo viola o encapsulamento. Identifique por quê e proponha uma refatoração que respeite os princípios da OOP.

```
public class Funcionario {
    public String nome;
    public double salario;
}
// uso:
Funcionario f = new Funcionario();
f.salario = -5000; // hmm...
```

Dica: Atributos public + ausência de validação = problema.

GABARITO

Errar é parte do processo! O objetivo do caderno é identificar onde você precisa revisar. Compare suas respostas com as soluções a seguir.

Resposta 1

int armazena números inteiros sem casas decimais (idade, quantidade, índice). double armazena números com casas decimais (preço, altura, média). Use int para contagens e double para grandezas contínuas. Cuidado: 10 / 3 com int dá 3, mas 10.0 / 3 com double dá 3.333...

Resposta 2

```
import java.util.Scanner;

public class Saudacao {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Digite seu nome: ");
        String nome = s.nextLine();
        System.out.println("Ola, " + nome + "!");
        s.close();
    }
}
```

Resposta 3

Imprime OK. A condição $x > 5 \ \&\& \ x < 20$ é verdadeira para $x = 10$ (10 é maior que 5 E menor que 20). O operador $\&\&$ exige que AMBAS as condições sejam verdadeiras.

Resposta 4

```
for (int i = 0; i <= 20; i += 2) {
    System.out.println(i);
}
// Alternativa: usar i % 2 == 0 dentro do laço
```

Resposta 5

- a) FALSO — a classe é apenas a definição; só os objetos (instâncias) ocupam memória.
- b) VERDADEIRO — cada objeto tem cópia própria dos atributos, com valores independentes.
- c) VERDADEIRO — sem new, não há instanciação (exceto para tipos primitivos, que não são classes).
- d) FALSO — se você não declara construtor, Java gera automaticamente um construtor padrão sem argumentos.

Resposta 6

```
public class Livro {
    String titulo;
    int    paginas;

    public String getTitulo() { return titulo; }
    public int    getPaginas() { return paginas; }
}
```

Resposta 7

```
public static void main(String[] args) {
```

```
Cachorro rex = new Cachorro();
Cachorro bidu = new Cachorro();
rex.nome = "Rex";
bidu.nome = "Bidu";
rex.latir();    // Rex disse Au!
bidu.latir();   // Bidu disse Au!
}
```

Cada objeto tem seu próprio atributo nome, independente do outro.

Resposta 8

Erro: a variável `a` foi declarada, mas o objeto nunca foi instanciado com `new`. Quando o código tenta acessar `a.nome`, a referência é `null`, gerando `NullPointerException`. Correção:

```
Aluno a = new Aluno();
a.nome = "Maria";
System.out.println(a.nome);
```

Resposta 9

```
public class ContaBancaria {
    private String numero;
    private double saldo;

    public ContaBancaria(String numero) {
        this.numero = numero;
        this.saldo = 0.0;
    }
    public void depositar(double valor) {
        saldo += valor;
    }
    public double getSaldo() { return saldo; }
}
```

Resposta 10

Saída: `false`. Justificativa: o estado de `ligada` começa `false`; o primeiro `ligar()` coloca em `true`; o segundo `ligar()` mantém em `true` (idempotente); o `apagar()` volta para `false`. O `isLigada()` devolve o último valor — `false`.

Resposta 11

```
public void setIdade(int idade) {
    if (idade < 0 || idade > 130) {
        throw new IllegalArgumentException(
            "Idade invalida: " + idade);
    }
    this.idade = idade;
}
```

Lançar exceção é geralmente a melhor opção porque alerta quem chamou que o valor está errado — falhar silenciosamente é pior, pois o bug fica escondido. Esse é exatamente o tipo de validação que justifica o uso de `setter` em vez de acesso direto ao atributo.

Resposta 12

Violação: os atributos são `public`, permitindo que qualquer parte do código altere o salário diretamente para um valor inválido (como `-5000`). Refatoração encapsulada:

```
public class Funcionario {
    private String nome;
    private double salario;

    public String getNome() { return nome; }
    public double getSalario() { return salario; }
    public void setNome(String nome) { this.nome = nome; }
    public void setSalario(double salario) {
        if (salario < 0) {
```

```
        throw new IllegalArgumentException(  
            "Salario nao pode ser negativo.");  
    }  
    this.salario = salario;  
}  
}
```

SENAC Entra21 — Módulo 07: Programação Orientada a Objetos — Material de apoio ao aluno